K L UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PROJECT REPORT

ON

"FOOD DONATION MANAGEMENT PLATFORM"

A Project Report Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Register Number	Name
2520030575	M. Likhitha
2520030198	K. Vaishnavi
2520030246	S. Nikhil

Under the Esteemed Guidance of

Dr. S. Spandana

Assistant Professor

Department of Computer Science and Engineering

Academic Year 2026–2027

K L (Deemed to be) University, Hyderabad, Telangana

K L UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

We hereby declare that the project report entitled “Food Donation Management Platform” is a record of bonafide work carried out by the undersigned students in partial fulfillment of the requirements for the award of the Bachelor of Technology degree in Computer Science and Engineering at K L (Deemed to be) University.

The work presented in this report is based on the Food Donation Management Platform source project supplied for documentation. The report describes the application structure, user roles, implementation, database entities, and workflows found in the supplied project files. To the best of our knowledge, this report has not been submitted, in whole or in part, to any other university or institution for the award of any degree or diploma.

Student Name	Register Number	Signature
M. Likhitha	2520030575	_____Likhitha_____
K. Vaishnavi	2520030198	_____Vaishnavi_____
S. Nikhil	2520030246	_____Nikhil_____

K L (DEEMED TO BE) UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project report entitled “Food Donation Management Platform” is a bonafide work carried out by M. Likhitha (2520030575), K. Vaishnavi (2520030198), and S. Nikhil (2520030246) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering, K L (Deemed to be) University, during the academic year 2026–2027.

The work embodied in this report has been completed under the guidance and supervision of the undersigned.

Signature of the Guide	Signature of the Course Coordinator	Signature of the HOD
_____	_____	_____
Name: Dr. S. Spandana	Name: _____	Name: _____

ACKNOWLEDGEMENT

We express our sincere gratitude to K L (Deemed to be) University and the Department of Computer Science and Engineering for providing the facilities and academic environment required to complete this project.

We are thankful to our project guide, Dr. S. Spandana, for the guidance, encouragement, and valuable suggestions provided during the development of the Food Donation Management Platform. We also thank the Course Coordinator and the Head of the Department for their support.

We extend our thanks to the teaching and non-teaching staff, our classmates, friends, and family members for their direct and indirect assistance. Their support helped us complete the project work and prepare this report.

The documentation was prepared by studying the supplied FoodBridge source archive, including its README file, React frontend, Spring Boot backend, entity classes, repositories, controllers, security configuration, and database settings.

M. Likhitha | K. Vaishnavi | S. Nikhil

ABSTRACT

Food wastage and food insecurity are two problems that affect communities in different ways. Restaurants, institutions, and other food providers may have surplus food that is still suitable for distribution, while NGOs and community organisations may need food for people who require support. When donors, NGOs, and volunteers coordinate through separate calls or messages, it can become difficult to share accurate information, arrange collection, and follow the delivery status.

The Food Donation Management Platform is a web-based application designed to bring these activities into one place. The application provides three user roles: Donor, NGO, and Volunteer. Donors can create food donation records by entering the food name, category, quantity, food type, description, preparation time, expiry time, and pickup location. NGOs can browse available donations and submit requests for a required quantity and number of people to serve. The donor who published the donation can approve or reject the request and assign a registered volunteer.

Volunteers can view assignments addressed to their account, accept an assignment, mark the food as picked up, and confirm delivery. After a delivery is completed, the application records the distributed quantity and the number of people served. The dashboard displays role-based counts such as donations, available packages, pending requests, completed distributions, food distributed in kilograms, and people served.

The frontend is developed using React and Vite. The backend is implemented using Java 17 and Spring Boot 3.4.5. Spring Data JPA is used for database access, Spring Security is configured for stateless authentication, BCrypt is used for password hashing, and JWT is used for signed login tokens. MySQL is used for persistent storage.

The platform aims to make food donation coordination more organised by connecting donation publication, NGO requests, donor decisions, volunteer assignment, and delivery confirmation in a single workflow.

INDEX

Section	Contents
Preliminary	Declaration, Certificate, Acknowledgement, Abstract
Chapter 1	Introduction – background, problem statement, scope, objectives, importance and users
Chapter 2	System Requirements – hardware, software and development tools
Chapter 3	Technology Stack – frontend, backend, database, APIs and libraries
Chapter 4	System Architecture – layers, request flow and deployment architecture
Chapter 5	System Design – modules, ER model, entities and business rules
Chapter 6	Implementation – frontend, backend, authentication and API integration
Chapter 7	Features – Donor, NGO, Volunteer, dashboard and distribution tracking
Chapter 8	Testing – approach, test cases and integration checks
Chapter 9	Deployment and Execution – prerequisites, database setup and run instructions
Chapter 10	Challenges and Limitations
Chapter 11	Future Enhancements
Chapter 12	Conclusion
Chapter 13	References
Appendices	API summary, user guide, screenshots checklist and source structure

CHAPTER 1 – INTRODUCTION

1.1 Background of the Project

Food is prepared and supplied every day by restaurants, hotels, canteens, events, and other food providers. At the end of a meal service or event, some prepared food may remain unused. At the same time, NGOs and community organisations work with people who may need food assistance. A structured connection between these two groups can help surplus food reach recipients instead of being discarded.

Food donation involves more than publishing a message that food is available. The donor needs to provide details about the food, quantity, preparation time, expiry time, and pickup address. An NGO must request a suitable quantity. The donor must decide whether to approve the request, and a volunteer may be needed to collect and deliver the food. Each step should be recorded so that the people involved can understand the current status.

The Food Donation Management Platform was developed around this process. It provides a shared web application for Donors, NGOs, and Volunteers. The frontend is built using React and Vite, while the backend uses Java and Spring Boot. MySQL stores user accounts, donations, requests, assignments, and distribution records.

1.2 Problem Statement

The main problem addressed by this project is the difficulty of coordinating surplus food donations through separate and informal communication channels. When details are shared manually, information about available quantity, food expiry, request approval, volunteer assignment, and delivery progress may not remain in one place.

- Donors need a clear way to publish food details and manage requests for their own donations.
- NGOs need to discover currently available food and request a quantity suitable for their requirements.
- Donors need to approve or reject requests made against their donations.
- Volunteers need to know which delivery has been assigned to them and update its progress.
- The people involved need to see whether food is available, reserved, picked up, or distributed.
- Completed deliveries should be reflected in the platform's food and people-served totals.

1.3 Scope of the Project

The project scope covers account registration and login, role-based access, donation creation, available-food browsing, NGO food requests, donor decisions, volunteer assignment, delivery status updates, role-specific dashboards, and distribution impact summaries. The application is intended for browser-based use and runs locally with a React development server, a Spring Boot backend, and a MySQL database.

The supplied project README identifies exactly three user-facing account roles: Donor, NGO, and Volunteer. It states that there is no Admin account in the application workflow. Although an ADMIN value is present in the backend role enum and some backend branches, public registration rejects ADMIN and the frontend registration form offers only the three roles described above.

1.4 Objectives

1. Provide a central platform where donors can publish details of surplus food.
2. Allow NGOs to browse available donations and request a required quantity.
3. Give the donor who owns a donation control over approval or rejection of requests.
4. Allow donors to assign registered volunteers to approved requests.
5. Enable assigned volunteers to accept, pick up, and complete deliveries in sequence.
6. Maintain records of distributed food quantity and the number of people served.
7. Display role-specific dashboard information using records stored in the database.
8. Protect user accounts and restrict access to data according to the signed-in role.

1.5 Importance of the Application

The application provides a common place to record donation and delivery information. Donors can manage the food they publish, NGOs can see available food from different donors, and volunteers can focus on their assigned deliveries. The request and assignment status values make it easier to identify the stage of a food handover. The dashboard also provides a simple view of food distributed and people served.

1.6 Target Users / Audience

User Role	Main Responsibilities
Donor	Register and log in; publish food donations; view own donations and related requests; approve or reject requests; assign a registered volunteer.
NGO	Register and log in; browse available donations; search and filter food packages; request a quantity; track own requests and delivery outcomes.
Volunteer	Register and log in; view assignments linked to the account; accept an assignment; mark pickup; confirm delivery.

CHAPTER 2 – SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The project is a browser-based full-stack application. The following configuration is a practical recommendation for development. The source archive does not enforce a particular processor or memory size.

Component	Recommended Configuration
Processor	Intel Core i5 / AMD Ryzen 5 or equivalent multi-core processor
RAM	8 GB minimum; 16 GB recommended when running the IDE, MySQL, backend and frontend together
Storage	At least 5 GB free for source code, dependencies, build output and database files; SSD preferred
Display	1366 × 768 or higher resolution
Network	Internet connection for downloading Maven and npm dependencies and accessing documentation
End-user device	Desktop, laptop, tablet or smartphone with a modern web browser

2.2 Software Requirements

Software	Version / Purpose
Operating System	Windows 10/11, Ubuntu Linux or macOS
Java JDK	Java 17 or newer for the Spring Boot backend
Maven	Maven 3.9 or newer for backend dependency management and execution
Node.js	Node.js 18 or newer for frontend tooling
npm	Package manager used with Node.js
MySQL Server	MySQL 8.x relational database
Web Browser	Google Chrome, Microsoft Edge or Mozilla Firefox
IDE / Editor	Visual Studio Code or IntelliJ IDEA

2.3 Development Tools

- Visual Studio Code or IntelliJ IDEA – editing and debugging project files.
- Maven – building and running the Spring Boot backend.
- npm and Vite – installing frontend dependencies and running the React development server.
- MySQL Workbench or MySQL command line – creating and inspecting the database.
- Browser Developer Tools – checking UI behavior, console messages and network requests.
- Postman or curl – manually checking REST API endpoints.

CHAPTER 3 – TECHNOLOGY STACK

3.1 Front-End Design

The frontend is built using React and Vite. React is used to divide the interface into components and manage changing application data. Vite provides the development server and build process. The main application is defined in App.jsx, the entry point is main.jsx, and the interface styling is maintained in style.css.

The frontend includes an authentication screen and a role-based workspace. The main workspace contains Dashboard, Food Donations, Food Requests, Assignments, People & NGOs, and Analytics & Reports sections in the source. The navigation items shown to a signed-in user are filtered according to the account role. Recharts is used for dashboard and report charts, while Lucide React supplies icons.

3.2 Back-End Design

The backend is implemented using Java 17 and Spring Boot 3.4.5. Spring Web is used to expose REST controllers. Spring Data JPA provides repository-based database access, and Hibernate maps Java entity classes to relational tables. Spring Security is configured to use stateless authentication. The backend source is organised into controller, DTO, model, repository, security, and service packages.

3.3 Database Design

MySQL is used as the persistent relational database. The configured database name is food_donation_db. The application uses JPA entity mappings and the setting spring.jpa.hibernate.ddl-auto=update. The principal entity classes are User, Donation, FoodRequest, Assignment, and Distribution. Relationships connect the donor to donations, NGOs to food requests, volunteers to assignments, and completed requests to distribution records.

3.4 APIs and External Libraries

Technology / Library	Purpose in the Project
Spring Web	REST API controllers and HTTP request/response handling
Spring Data JPA / Hibernate	Entity mapping and database operations
Spring Security	Stateless request security and authenticated API access
BCryptPasswordEncoder	Password hashing before account details are stored
JJWT 0.12.6	Creation and validation of signed JSON Web Tokens
MySQL Connector/J	Connection between the Java backend and MySQL
React / Vite	Frontend component rendering and local development server
Recharts	Area and bar charts in the dashboard and analytics screen
Lucide React	Icons used in the user interface

3.5 Version Control

The supplied archive contains the source project and a .gitignore file. Git can be used to maintain source history and GitHub can be used for team collaboration. The supplied ZIP does not provide repository history or remote repository details, so no existing remote repository is claimed in this report.

CHAPTER 4 – SYSTEM ARCHITECTURE

4.1 High-Level Architecture

The Food Donation Management Platform follows a three-layer web architecture. The presentation layer is the React application running through Vite. The application layer is the Spring Boot REST backend, which handles authentication, role checks, request processing, and business rules. The data layer is MySQL, accessed through Spring Data JPA repositories.

PRESENTATION LAYER	APPLICATION LAYER	DATA LAYER
React + Vite App.jsx / components style.css	Spring Boot 3.4.5 REST Controllers Services / Security	MySQL 8 food_donation_db JPA Entities / Tables

Browser UI ⇄ HTTP / JSON REST API ⇄ Spring Boot ⇄ Spring Data JPA ⇄ MySQL

4.2 Architecture Diagram – Request Direction

Step	Description
1. User interface	The user signs in and opens a role-specific page in the React application.
2. API request	The frontend sends an HTTP request to an endpoint under /api. Protected requests include an Authorization: Bearer token header.
3. Security filter	JwtAuthFilter reads the token, validates it through JwtService, and loads the matching user.
4. Controller	The relevant REST controller receives the request and checks the current user's role and ownership.
5. Repository	Spring Data JPA repositories read or update the corresponding MySQL records.
6. Response	The backend returns JSON data or an HTTP error response, and the frontend refreshes the displayed information.

The supplied Vite configuration proxies /api requests to http://localhost:8080 during local development.

4.3 Description of Each Layer

Presentation Layer

The presentation layer displays the login and registration form, dashboard cards, donation listings, food request table, assignment table, and analytics views. It also provides forms for entering donation information and controls for approving requests, assigning volunteers, and updating delivery status. React state is used to update the interface after API actions.

Application / Business Layer

The Spring Boot backend receives API requests through controllers. AuthController handles registration and login. DonationController manages donation records and availability. RequestController handles NGO requests and donor decisions. AssignmentController creates volunteer assignments and controls the delivery status sequence. DashboardController calculates role-specific summary values.

Data Layer

The repository interfaces extend Spring Data JPA's JpaRepository. They provide database operations for users, donations, food requests, assignments, and distributions. Entity classes define the fields and relationships that Hibernate maps to MySQL.

4.4 Request Flow

9. A donor, NGO, or volunteer opens the React application and signs in.
10. The frontend sends email and password details to POST /api/auth/login.
11. The backend checks the stored password hash and returns a signed JWT with user details.
12. The frontend stores the token and sends it in the Authorization: Bearer header for protected API requests.
13. JwtAuthFilter verifies the token and sets the authenticated user in the Spring Security context.
14. The controller applies role and ownership rules before performing an operation.
15. The repository saves or retrieves the required records from MySQL.
16. The backend returns a response and the frontend reloads the current dashboard data.

4.5 Deployment Architecture

The supplied project is configured for local development. The backend runs at http://localhost:8080. The Vite frontend runs at the local URL printed by the npm run dev command, usually http://localhost:5173. MySQL is configured on localhost:3306. The frontend Vite configuration forwards requests beginning with /api to the backend.

CHAPTER 5 – SYSTEM DESIGN

5.1 Main Modules

Module	Description
Authentication	Registration, login, password hashing, JWT generation and current-user identification.
Donation Management	Create food donation records and retrieve donations according to the signed-in user's role.
Food Request Management	Allow NGOs to request available food and allow the owning donor to approve or reject pending requests.
Volunteer Management	Retrieve registered volunteers for donor assignment and associate a volunteer with an approved request.
Assignment and Delivery Tracking	Allow the assigned volunteer to accept an assignment, mark pickup, and confirm delivery.
Dashboard	Display donation counts, available packages, pending requests, completed distributions, food distributed and people served.
Analytics and Reports	Show food quantities by category, request status counts, and provide CSV export for donation records in the frontend.

5.2 Role-Based Access

Role	Main Access in the Application
Donor	Own donation records, requests received for own donations, volunteer selection for approved requests, assignments related to own donations, and donor dashboard information.
NGO	All currently available donations, own submitted requests, related assignment details, and NGO dashboard information.
Volunteer	Only assignments assigned to the volunteer account and volunteer dashboard information.

The README describes exactly these three account roles. Public registration does not permit creation of an ADMIN account.

5.3 ER Diagram / Database Model

The data model contains five main entities. A User may act as a Donor, NGO, or Volunteer according to the role stored in the account. One donor can publish multiple donations. An NGO can submit multiple food requests, and each food request refers to one donation. An approved request can have one volunteer assignment. A completed delivery is represented by a distribution record.

ENTITY	RELATIONSHIP SUMMARY
USER	One donor can publish many donations. One NGO can submit many food requests. One volunteer can receive many assignments.
DONATION	Each donation belongs to one donor. A donation can be referred to by food requests.
FOOD_REQUEST	Each request belongs to one NGO and refers to one donation. An approved request can be assigned to a volunteer.
ASSIGNMENT	Each assignment refers to one food request and one volunteer. The assignment has a delivery status.
DISTRIBUTION	A distribution record refers to one food request and stores the distributed quantity and people served.

USER (DONOR) 1 ———< DONATION 1 ———< FOOD_REQUEST >——— 1 USER (NGO)

FOOD_REQUEST 1 ——— 0..1 ASSIGNMENT >——— 1 USER (VOLUNTEER)

FOOD_REQUEST 1 ——— 0..1 DISTRIBUTION

The relationships above represent the entity associations present in the supplied Java model. The database creates the corresponding foreign-key relationships through JPA mappings.

5.4 Main Database Entities and Attributes

Entity	Important Attributes
users	id (PK), name, email (unique), password, role, phone, organization, address, verified, created_at
donation	id (PK), donor_id (FK), food_name, category, quantity_kg, food_type, description, prepared_at, expiry_at, pickup_address, city, status, created_at
food_requests	id (PK), ngo_id (FK), donation_id (FK), requested_kg, people_to_serve, purpose, urgency, status, priority_score, created_at
assignment	id (PK), food_request_id (FK), volunteer_id (FK), status, assigned_at, completed_at
distribution	id (PK), food_request_id (FK), distributed_kg, people_served, proof_reference, distributed_at

5.5 Status Values

Record	Status Values
Donation	AVAILABLE, RESERVED, PICKED_UP, DISTRIBUTED, EXPIRED, CANCELLED
Food Request	PENDING, APPROVED, REJECTED, CANCELLED
Assignment	PENDING, ACCEPTED, PICKED_UP, DELIVERED, CANCELLED

5.6 Important Business Rules

- Only a Donor can create a donation through the normal application workflow.
- Donation expiry must be in the future and must not be earlier than the preparation time.
- Only an NGO can submit a food request.
- The requested quantity must be positive and cannot exceed the donation quantity.
- Only the donor who published the donation can approve or reject the request.
- Only a pending request can be approved or rejected; approval reserves the donation.
- Only the donor who owns an approved request can assign a registered Volunteer.
- Only the assigned Volunteer can update the assignment status.
- A volunteer must accept an assignment before marking it as picked up, and must mark pickup before delivery completion.

CHAPTER 6 – IMPLEMENTATION

6.1 Front-End Implementation

The frontend source is located in the frontend directory. The application entry point is src/main.jsx, which renders App.jsx and imports style.css. App.jsx contains the main application state, authentication screen, navigation, dashboard, donation list, request list, assignment list, people section, analytics section, and donation form.

The interface uses a green and white visual theme. The login and registration screen introduces the purpose of the platform and provides account creation for Donor, NGO, and Volunteer. After login, the workspace displays navigation according to the user's role. A profile area shows the signed-in user's name and role, and a sign-out action clears the saved login information.

6.2 Main Frontend Sections

Page / Component	Responsibility
Auth	Displays login and registration forms and handles the authentication request.
Dashboard	Displays summary cards, donation activity chart, community impact values, recent donations and expiry alerts.
Donations	Displays food packages, search and status filters; NGOs can request available food and donors can cancel an available donation.
Requests	Displays requests; donors can accept/reject pending requests and assign volunteers to approved requests.
Assignments	Displays pickup and delivery assignments; volunteers update their own assignment status.
Analytics	Displays food distributed, people served, completed distributions, category quantities, request pipeline and CSV export.
DonationModal	Collects food name, category, food type, quantity, description, preparation time, expiry time and pickup location.

6.3 Frontend API Communication

The frontend uses a shared api helper to send fetch requests to endpoints under /api. The helper adds the Content-Type header and, when a token is available, adds the Authorization: Bearer token header. After successful actions, the application displays a short notification and reloads the dashboard, donation, request, and assignment data.

The frontend also refreshes data when the browser tab receives focus and every ten seconds while a user is signed in. This helps the interface show changes made by other users during the same local application session.

6.4 Back-End Implementation

The backend source is located in backend/src/main/java/com/foodbridge. FoodDonationApplication.java is the Spring Boot entry point. The project uses separate packages for DTO classes, entity models, repositories, controllers, security components, and the CurrentUser service.

Package / Class Group	Responsibility
model	Defines User, Donation, FoodRequest, Assignment, Distribution and the role/status enums.
repository	Provides database access for users, donations, food requests, assignments and distributions through Spring Data JPA.
controller	Contains AuthController, DonationController, RequestController, AssignmentController, DashboardController and UserController.
dto	Defines request data structures and validation rules for registration, login, donations and food requests.
security	Contains JwtService, JwtAuthFilter and SecurityConfig.
service	CurrentUser retrieves the authenticated account from the security context and user repository.

6.5 Controller Responsibilities

- AuthController: registers a user and authenticates login credentials.
- DonationController: lists donations based on role, creates donor donations, retrieves a donation and updates donation status.
- RequestController: lists requests for the signed-in NGO or donor, creates NGO requests, and processes donor decisions.
- AssignmentController: lists assignments according to role, creates volunteer assignments, and updates delivery progress.
- DashboardController: calculates donation, request, distribution, food quantity, and people-served summary values.
- UserController: provides registered volunteer details to donors for assignment.

6.6 Repository Layer

The repository interfaces extend JpaRepository and use method names to retrieve records by fields such as donor ID, NGO ID, volunteer ID, donation status, or request ID. This keeps common database queries separate from the controller code.

6.7 Authentication and Authorization

During registration, the backend checks whether the email address is already registered. The password is encoded using BCryptPasswordEncoder before the user record is saved. The registration endpoint accepts the account role, but rejects ADMIN account creation. The frontend registration form provides Donor, NGO, and Volunteer as the available choices.

During login, the backend finds the account using the email address and checks the submitted password against the stored BCrypt hash. If the credentials are valid, JwtService creates a signed JSON Web Token containing the email as the subject and the user's role as a claim. The configured token expiry is 86,400,000 milliseconds, which is 24 hours.

For protected API calls, the frontend sends the token in the Authorization header. JwtAuthFilter reads the bearer token, verifies it using JwtService, finds the matching user, and places the authenticated identity and role in the Spring Security context. The backend is configured for stateless sessions.

6.8 API Integration

The application exposes REST endpoints under the /api path. Most endpoints exchange JSON request and response data. The frontend uses the API responses to update its tables, dashboard cards, and status labels.

6.9 Validation and Error Handling

- Registration checks name, email format, password length and role input through DTO validation and controller checks.
- Donation details require food name, category, positive quantity, preparation time, expiry time and pickup address.
- Donation expiry must be later than the current time and cannot be before the preparation time.
- Food request details require a donation ID and positive requested quantity and people-to-serve values.
- Controller methods return HTTP error responses for unavailable donations, invalid quantities, wrong roles, and invalid status transitions.
- Authentication failures return an unauthorized response for invalid email or password.

The supplied source includes request validation and controller-level error responses. A separate global exception-handler package is not present in this project archive.

CHAPTER 7 – FEATURES

7.1 Donor Features

- Create an account as a Donor and sign in using email and password.
- Publish a food donation with food name, category, quantity in kilograms, food type, description, preparation time, expiry time, pickup address and city.
- View the donations published by the signed-in donor.
- Search and filter donation records by food name, category, city and status in the frontend.
- View requests received for the donor's own donations.
- Approve or reject a pending NGO request.
- Assign a registered volunteer to an approved request.
- View donor-specific dashboard counts and completed distribution impact.

7.2 NGO Features

- Create an account as an NGO and sign in.
- Browse currently available donations published by different donors.
- Search food packages and filter the displayed list by status.
- Submit a request for a required quantity and number of people to serve.
- Provide purpose and urgency details for a request.
- View only the requests submitted by the signed-in NGO.
- Track request and assignment status for its own food requests.
- View NGO-specific dashboard information about available food and completed distributions.

7.3 Volunteer Features

- Create an account as a Volunteer and sign in.
- View only assignments linked to the volunteer's account.
- Accept a pending delivery assignment.
- Mark the food as picked up after accepting the assignment.
- Mark the delivery as completed after pickup.
- View completed delivery counts and impact values on the dashboard.

7.4 Food Request Priority

When an NGO submits a request, the backend stores an urgency value. The initial priority score is based on the urgency selection: HIGH receives 30 points, MEDIUM receives 20 points, and other values receive 10 points. The backend also checks the donation expiry time. If fewer than 24 hours remain, 40 points are added; if fewer than 48 hours remain, 20 points are added. The calculated score is stored with the food request.

The request table in the frontend displays the urgency and priority score with a visual priority label. This gives the donor additional information when reviewing requests.

7.5 Assignment and Delivery Workflow

PENDING → ACCEPTED → PICKED_UP → DELIVERED

A donor can create an assignment only after the NGO request has been approved. The selected account must have the Volunteer role, and the request must not already have an assignment. The volunteer assigned to the request is the only account permitted to update its delivery status.

When the volunteer marks the assignment as PICKED_UP, the related donation status is changed to PICKED_UP. When the volunteer marks it as DELIVERED, the assignment completion time is recorded, the donation status changes to DISTRIBUTED, and a Distribution record is saved with the requested kilograms and people-to-serve value.

7.6 Dashboard and Reports

The dashboard summary is calculated from database records. Depending on the signed-in role, it shows total donations or assignments, available packages or pending assignments, pending requests, completed distributions, food distributed in kilograms, and people served. The community impact section also displays counts of donors, NGOs, and volunteers.

The Analytics & Reports section displays food distributed, people served, completed distributions, open requests, quantities by food category, and the request pipeline. The frontend provides a CSV export for the donation records currently loaded in the page.

The frontend source contains a People & NGOs section and a verification action, but the supplied backend UserController exposes only the volunteer-list endpoint. The section is not included in the role-filtered navigation. It is therefore not described as a completed, working user-management feature.

CHAPTER 8 – TESTING

8.1 Testing Approach

Testing should cover registration, login, role-based access, donation creation, food request validation, donor decisions, volunteer assignment, delivery status transitions, dashboard calculations, and database persistence. The supplied archive contains application source and local run instructions, but it does not include a formal test execution report or recorded test evidence. The following cases form a recommended test plan and should be executed on the local setup.

8.2 Test Cases

ID	Test Scenario	Expected Result
TC-01	Register a new Donor with valid details.	Account is created and a success response is returned.
TC-02	Register using an email already stored.	Registration is rejected with an email-already-registered message.
TC-03	Log in with valid credentials.	JWT token and user details are returned.
TC-04	Log in with an incorrect password.	Login is rejected with an authentication error.
TC-05	Donor creates a donation with a future expiry.	Donation is saved with AVAILABLE status.
TC-06	Create a donation with expiry in the past.	Request is rejected.
TC-07	NGO browses available donations.	Currently available donations are displayed.
TC-08	NGO requests a positive quantity within available quantity.	Food request is created with PENDING status and priority score.
TC-09	NGO requests more than the available quantity.	Request is rejected.
TC-10	Owning donor approves a pending request.	Request becomes APPROVED and donation becomes RESERVED.
TC-11	Another donor attempts to decide the request.	Request is rejected as forbidden.
TC-12	Donor assigns a registered volunteer to an approved request.	Assignment is created with PENDING status.
TC-13	Assigned volunteer accepts, picks up and delivers.	Assignment reaches DELIVERED and distribution is recorded.

8.3 Additional Test Cases

ID	Test Scenario	Expected Result
TC-14	Unassigned volunteer tries to update an assignment.	Request is rejected.
TC-15	Volunteer marks PICKED_UP before ACCEPTED.	Request is rejected because acceptance is required first.
TC-16	Volunteer marks DELIVERED before PICKED_UP.	Request is rejected because pickup is required first.
TC-17	Volunteer completes a valid delivery.	Completion time and distribution details are stored.
TC-18	Donor opens donation list.	Only donations published by that donor are returned.
TC-19	NGO opens donation list.	Only available donations are returned.
TC-20	Volunteer opens assignment list.	Only assignments for that volunteer are returned.
TC-21	Dashboard is opened under different roles.	Summary values reflect the signed-in role.
TC-22	Donation data is exported from Analytics.	A CSV file is generated by the frontend.

8.4 Tools Used for Testing

- Browser and browser developer tools for checking pages, forms and UI updates.
- Postman or curl for sending REST API requests.
- MySQL Workbench or MySQL CLI for checking saved records.
- Maven commands for backend compilation and execution.
- npm run build for frontend production compilation.

8.5 Integration Testing

Integration testing should verify the complete path from the React interface to the Spring Boot controller, service logic, repository, and MySQL database. A practical demonstration should use one account for each role. The donor publishes food, the NGO submits a request, the donor approves and assigns a volunteer, and the volunteer completes the delivery. The dashboard and request records should then reflect the completed distribution.

CHAPTER 9 – DEPLOYMENT AND EXECUTION

9.1 Prerequisites

- Java JDK 17 or newer.
- Maven 3.9 or newer.
- Node.js 18 or newer and npm.
- MySQL Server 8.x running locally, normally on port 3306.

9.2 Database Configuration

The backend database configuration is stored in backend/src/main/resources/application.properties. The JDBC URL uses localhost:3306 and the database name food_donation_db. The default configuration uses the MySQL username root and password root. If the local MySQL credentials are different, update DB_USERNAME and DB_PASSWORD environment variables or edit the properties file. The application uses spring.jpa.hibernate.ddl-auto=update.

Create the database if it does not already exist:

```
CREATE DATABASE food_donation_db;
```

9.3 Run the Backend

Open a terminal in the project folder and move into the backend directory. Run the Spring Boot application using Maven:

```
cd backend
mvn spring-boot:run
```

The backend is configured to run at http://localhost:8080.

9.4 Run the Frontend

Open a second terminal and move into the frontend directory. Install the npm dependencies and start the Vite development server:

```
cd frontend
npm install
npm run dev
```

Open the local URL printed by Vite, usually http://localhost:5173. The Vite configuration proxies /api requests to the backend at port 8080.

9.5 Suggested Demonstration Sequence

17. Start MySQL and confirm that the food_donation_db database is available.
18. Start the Spring Boot backend and wait for the application to complete startup.
19. Start the Vite frontend in a second terminal and open the local browser URL.
20. Register or sign in using a Donor account and publish a food donation with a future expiry time.
21. Sign out and sign in using an NGO account. Browse the available donation and submit a request.
22. Sign out and return to the Donor account. Open Food Requests and approve the pending request.
23. Assign a registered Volunteer account to the approved request.
24. Sign out and sign in using that Volunteer account. Accept the assignment and update it to PICKED_UP.
25. Update the assignment to DELIVERED and confirm the completed status.
26. Review the related request, donation status, distribution details and dashboard impact values.

9.6 Configuration and Safety Notes

- Do not use the demonstration database password or JWT secret in a public deployment.
- Set a strong random JWT_SECRET through an environment variable before production use.
- Use a strong database password and restrict database access.
- Configure CORS for the actual frontend origin and use HTTPS in a deployed environment.
- Back up the database before changing project versions or database settings.
- The README states that the supplied source has not been fully end-to-end tested against the local Windows/MySQL setup. Verify the complete workflow locally.

CHAPTER 10 – CHALLENGES AND LIMITATIONS

10.1 Development Challenges Addressed by the Design

- Coordinating three different roles: the application provides separate views and backend checks for Donors, NGOs and Volunteers.
- Keeping donation records private to the owner: donor listing is scoped to the signed-in donor, while NGOs browse currently available donations.
- Preventing invalid food requests: the backend checks that the donation is available and that the requested quantity is positive and does not exceed the listed quantity.
- Controlling request decisions: only the donor who published the donation can approve or reject its request.
- Maintaining delivery order: the volunteer must accept before pickup and must mark pickup before delivery completion.
- Recording distribution impact: a completed delivery creates a distribution record containing kilograms distributed and people served.
- Protecting account access: BCrypt password hashing and signed JWT authentication are used for account security.

10.2 Current Limitations

- The supplied project is configured for local development; a completed hosted deployment is not documented.
- The README states that full end-to-end testing against the local Windows/MySQL setup is still required.
- No live GPS tracking or map-based route navigation is present in the supplied source.
- No integrated online payment gateway is present.
- The application records food details and expiry time, but a dedicated food-quality inspection or certification workflow is not documented.
- Email, SMS, and push notifications are not implemented in the supplied source.
- The People & NGOs screen has frontend code, but the backend endpoint required for the displayed verification action is not present in the archive.
- The ADMIN enum and some backend branches exist, but public registration and the documented workflow support only Donor, NGO and Volunteer accounts.

10.3 Safeguards in the Current Code

The current implementation uses password hashing, signed token verification, stateless API authentication, role checks, ownership checks, input validation, donation expiry checks, quantity checks, and ordered assignment status transitions. These controls support the intended workflow. They do not replace the need for complete testing, production security review, secure secret management, and deployment configuration.

CHAPTER 11 – FUTURE ENHANCEMENTS

The current application provides the main donation-to-delivery workflow. The following improvements can be considered in later versions. These are proposed extensions and are not represented as completed features in the supplied source archive.

- Email and SMS notifications for new donation requests, donor decisions, volunteer assignments, pickup reminders and delivery confirmation.
- Live location sharing so NGOs and donors can follow a volunteer during collection and delivery.
- Map integration to help volunteers find pickup and drop-off locations.
- Food safety verification with checklists for preparation time, storage conditions, packaging and expiry.
- Image upload so donors can attach photographs of donated food packages.
- Advanced matching that recommends donations to NGOs based on location, quantity, urgency and expiry time.
- More detailed analytics for food saved, average response time, successful deliveries and community reach.
- Mobile application or progressive web application for volunteers who need to update deliveries while travelling.
- Automated database backups, application monitoring and structured activity logs.
- Expanded automated unit, integration and security testing.

11.1 Expected Benefits of the Enhancements

Notifications can reduce delays in responding to requests. Location features can help volunteers plan collection and delivery. Food safety checks can improve the quality of information available to NGOs. Matching and analytics can help the platform use donation records more effectively and provide a clearer picture of its community impact.

CHAPTER 12 – CONCLUSION

The Food Donation Management Platform provides a structured way to coordinate surplus food donations between Donors, NGOs, and Volunteers. It brings donation publication, NGO requests, donor approval, volunteer assignment, pickup confirmation, and delivery completion into one application. The use of separate account roles helps each user work with the information related to their responsibilities.

The frontend is developed using React and Vite, and the backend uses Java 17 with Spring Boot 3.4.5. Spring Data JPA connects the application to MySQL. The system uses BCrypt password hashing and JWT-based authentication to protect account access. The entity model stores user, donation, food request, assignment, and distribution information.

The project also demonstrates how a workflow can be represented through status values. A food request begins as PENDING and can be APPROVED or REJECTED by the donor who owns the donation. An approved request can be assigned to a volunteer. The volunteer then moves the assignment through ACCEPTED, PICKED_UP, and DELIVERED. A completed delivery is recorded as a distribution and contributes to the dashboard's food and people-served totals.

The project demonstrates practical application of frontend development, REST API design, Java backend programming, relational database mapping, authentication, role-based access, and workflow validation. With complete local testing, security hardening, and the proposed future enhancements, the platform can be extended to support a wider food donation network.

12.1 Skills and Concepts Applied

- React component development and frontend state management.
- Vite-based frontend development and build process.
- Java 17 and Spring Boot REST API development.
- Spring Data JPA and Hibernate entity mapping.
- MySQL database design and relational associations.
- Password hashing, JWT generation and token validation.
- Role-based access and record ownership checks.
- Input validation, workflow status management and dashboard calculations.

CHAPTER 13 – REFERENCES

27. Food Donation Management Platform – README.md included in the supplied source archive.
28. Food Donation Management Platform – frontend/src/App.jsx and frontend/src/style.css.
29. Food Donation Management Platform – backend/src/main/java/com/foodbridge model, controller, repository, security and DTO packages.
30. Food Donation Management Platform – backend/src/main/resources/application.properties.
31. Food Donation Management Platform – backend/pom.xml.
32. Spring Boot Reference Documentation – https://docs.spring.io/spring-boot/
33. Spring Data JPA Reference Documentation – https://docs.spring.io/spring-data/jpa/reference/
34. Spring Security Reference Documentation – https://docs.spring.io/spring-security/reference/
35. React Documentation – https://react.dev/
36. Vite Documentation – https://vite.dev/guide/
37. MySQL 8.0 Reference Manual – https://dev.mysql.com/doc/
38. Apache Maven Documentation – https://maven.apache.org/guides/
39. JSON Web Token introduction – https://jwt.io/introduction

APPENDICES

Appendix A – REST API Summary

The following table summarises the main API routes identified in the supplied controller classes. Protected routes require a valid bearer token.

Area	Method and Endpoint	Purpose / Access
Authentication	POST /api/auth/register	Create a Donor, NGO or Volunteer account.
Authentication	POST /api/auth/login	Authenticate user and return token and profile.
Donations	GET /api/donations	List donations according to signed-in role; optional status and q filters.
Donations	POST /api/donations	Create a donation; Donor role.
Donations	GET /api/donations/{id}	Retrieve a donation subject to role and ownership checks.
Donations	PUT /api/donations/{id}/status	Update donation status; owner donor.
Requests	GET /api/requests	List requests for the signed-in NGO or donor.
Requests	POST /api/requests	Submit a food request; NGO role.
Requests	PUT /api/requests/{id}/decision	Approve or reject a pending request; owning donor.
Assignments	GET /api/assignments	List assignments according to role.
Assignments	POST /api/assignments?requestId=&volunteerId=	Assign a registered volunteer to an approved request; donor.
Assignments	PUT /api/assignments/{id}/status	Update assignment status; assigned volunteer.
Users	GET /api/users/volunteers	List volunteers for donor assignment; Donor role.
Dashboard	GET /api/dashboard/summary	Return role-specific dashboard summary values.

Appendix B – User Guide

Donor Workflow

40. Register using the Donor account type and sign in.
41. Open Food Donations and choose Add donation.
42. Enter food name, category, quantity, food type, preparation time, expiry time and pickup address.
43. Open Food Requests to review requests submitted against your donations.
44. Accept or reject a pending request. For an approved request, select a registered volunteer.

NGO Workflow

45. Register using the NGO / Organization account type and sign in.
46. Open Food Donations to browse currently available food packages.
47. Search for a suitable package and submit the required quantity.
48. Open Food Requests to track the status of your submitted request and related assignment.

Volunteer Workflow

49. Register using the Volunteer account type and sign in.
50. Open Assignments to view deliveries assigned to your account.
51. Accept the pending assignment.
52. After collecting the food, update the status to PICKED_UP.
53. After completing the handover, update the status to DELIVERED.

Appendix C – Screenshots to Insert

The supplied ZIP contains application source files but does not include captured runtime screenshots. The following screenshots should be taken after running the application locally and inserted in the final printed report.

- Login and registration page showing the three account types.
- Donor dashboard and Add Donation form.
- Donor Food Donations page and Food Requests page.
- NGO Food Donations page and request submission flow.
- Volunteer Assignments page showing status updates.
- Analytics & Reports page showing dashboard values and charts.

Appendix D – Source Project Structure

```
FoodBridge-Three-Role-Full-Project-FIXED/
├── README.md
├── frontend/
│   ├── package.json
│   ├── vite.config.js
│   └── src/
│       ├── App.jsx
│       ├── main.jsx
│       └── style.css
└── backend/
    ├── pom.xml
    └── src/main/
        ├── java/com/foodbridge/
        │   ├── FoodDonationApplication.java
        │   ├── controller/
        │   ├── dto/
```

```
│   ├── model/
│   ├── repository/
│   ├── security/
│   └── service/
└── resources/application.properties
```